

Travail Pratique du GROUPE 2

Vue d'ensemble du fonctionnement de l'application

L'application **Colon Cancer Classifier** est une interface web développée avec Django qui permet :

1. **Saisie des données** : Les utilisateurs entrent des informations sur des patients (nom, âge, sexe) et des valeurs de caractéristiques (expression du gène UGP2).
2. **Prédiction** : Un modèle de machine learning (Logistic Regression, entraîné dans le notebook) prédit si le tissu est "normal" ou "tumoral" en fonction de la valeur UGP2.
3. **Affichage des résultats** : Les prédictions sont affichées avec des détails (valeur de la caractéristique, probabilité, statut).
4. **Historique** : Les utilisateurs peuvent consulter l'historique des prédictions pour tous les patients.

Le modèle de machine learning est entraîné dans le notebook Jupyter (tp_ml.ipynb) et sauvegardé sous forme de fichiers cc_model_lr.pkl (modèle) et scaler.pkl (scaler pour la normalisation). Ces fichiers sont utilisés dans l'application Django pour effectuer les prédictions.

1. Modelisation avec scikit-learn (tp_ml.ipynb)

Le notebook est responsable de l'entraînement du modèle de machine learning. Voici les étapes clés :

a. Importation des bibliothèques

- Bibliothèques pour le traitement des données : pandas, numpy, scipy.stats.
- Outils de machine learning : sklearn (modèles, prétraitement, métriques), joblib (sauvegarde du modèle).
- Modèles testés : Decision Tree, Random Forest, Logistic Regression, SVM, KNN, Naive Bayes, Gradient Boosting, Neural Network.

b. Chargement et prétraitement des données

- **Données** : Fichier colon_cancer.csv (804 échantillons), contenant des expressions géniques et une colonne tissue_status ("normal" ou "tumoral").
- **Prétraitement** :
 - tissue_status est converti en valeurs numériques : "normal" → 0, "tumoral" → 1.
 - Les valeurs manquantes sont supprimées (si présentes).
 - Les caractéristiques (gènes) sont séparées de la cible (tissue_status), et l'identifiant id_sample est exclu.

c. Sélection du meilleur gène

- Un test statistique (t-test) est effectué pour chaque gène afin de comparer les expressions entre les tissus normaux ($y == 0$) et tumoraux ($y == 1$).
- Le gène avec la p-value la plus faible est sélectionné : **UGP2** (p-value = 0.000000), indiquant une forte différence d'expression entre les deux classes.

d. Préparation des données pour la modélisation

- Seule la colonne UGP2 est utilisée comme caractéristique (X_best).
- Les données sont normalisées avec StandardScaler pour standardiser les valeurs (moyenne = 0, écart-type = 1).

Travail Pratique du GROUPE 2

- Les données sont divisées en ensembles d'entraînement (80%, 643 échantillons) et de test (20%, 161 échantillons) avec `train_test_split`.

e. Entraînement et évaluation des modèles

- Huit modèles sont testés via validation croisée (5 plis) sur l'ensemble d'entraînement :
 - Decision Tree : 0.9564
 - Random Forest : 0.9564
 - Logistic Regression : **0.9735** (meilleur)
 - SVM : 0.9720
 - KNN : 0.9688
 - Naive Bayes : 0.9688
 - Gradient Boosting : 0.9564
 - Neural Network : 0.9735
- **Logistic Regression** est sélectionné comme le meilleur modèle (précision moyenne = 0.9735).

f. Entraînement final et exportation

- Le modèle Logistic Regression est entraîné sur l'ensemble d'entraînement complet.
- Évaluation sur l'ensemble de test :
 - Précision : 1.0000
 - Précision, rappel, score F1 : 1.0000 (indiquant une séparation parfaite des classes sur les données de test).
- Le modèle et le scaler sont sauvegardés :
 - `cc_model_lr.pkl` : Modèle Logistic Regression.
 - `scaler.pkl` : Scaler pour normaliser les valeurs UGP2.

Rôle dans l'application : Ce notebook génère les fichiers `cc_model_lr.pkl` et `scaler.pkl`, qui sont utilisés dans l'application Django pour effectuer les prédictions en temps réel.

2. Modèles Django (`models.py`)

Les modèles définissent la structure des données dans la base de données :

- **Modèle Patient** :
 - **Champs** : `name` (chaîne, max 100 caractères), `age` (entier positif), `gender` (choix : 'M', 'F', 'O'), `created_at` (date automatique).
 - **Rôle** : Stocke les informations des patients saisies via le formulaire.
- **Modèle Prediction** :
 - **Champs** : `patient` (clé étrangère vers Patient), `feature_name` (nom de la caractéristique, max 50 caractères), `feature_value` (flottant, valeur UGP2), `prediction` (chaîne : "normal" ou "tumoral"), `probability` (flottant, pourcentage), `is_positive` (booléen : True pour normal, False pour tumoral), `created_at` (date automatique).
 - **Rôle** : Stocke les prédictions associées à un patient, avec leurs détails.
- **Relation** : Un patient peut avoir plusieurs prédictions (relation un-à-plusieurs via `related_name='predictions'`).

3. Vues Django (`views.py`)

Les vues gèrent la logique de l'application :

Travail Pratique du GROUPE 2

- **HomeView :**
 - **GET :** Affiche le formulaire pour saisir les informations des patients et des caractéristiques.
 - **POST :**
 - Collecte et valide les données des patients (nom, âge, sexe) et des caractéristiques (valeurs UGP2).
 - Crée un objet Patient pour chaque patient.
 - Normalise les valeurs UGP2 avec `scaler.pkl`, effectue des prédictions avec `cc_model_lr.pkl`, calcule les probabilités, et sauvegarde les prédictions (Prediction).
 - Affiche les résultats dans `results.html` ou retourne une erreur en cas d'échec.
- **PredictionHistoryView :**
 - **GET :** Récupère tous les patients et leurs prédictions, triés par date, et affiche l'historique dans `history.html`.

4. Templates Django

- **base.html :**
 - **Rôle :** Template de base définissant la structure commune (barre de navigation, corps, pied de page) avec Tailwind CSS et JavaScript personnalisé.
- **home.html :**
 - **Rôle :** Affiche un formulaire dynamique pour saisir les informations des patients (nom, âge, sexe) et des valeurs UGP2, avec des boutons pour ajouter/supprimer des patients ou des caractéristiques.
- **results.html :**
 - **Rôle :** Affiche les résultats des prédictions pour chaque patient, avec des cartes montrant la valeur UGP2, la prédiction ("Normal" ou "Tumoral"), la probabilité, et des boutons pour refaire une prédiction ou voir l'historique.
- **history.html :**
 - **Rôle :** Affiche un tableau avec l'historique des prédictions (nom, âge, sexe, caractéristique, valeur, prédiction, probabilité, statut, date), avec un bouton pour refaire une prédiction.

Points forts

1. **Précision du modèle :** Le modèle Logistic Regression atteint une précision de 1.0000 sur les données de test, suggérant une excellente séparation des classes basée sur UGP2.
2. **Interface conviviale :** Le formulaire dynamique et les pages de résultats/historique sont intuitifs.
3. **Gestion des erreurs :** Les vues valident rigoureusement les entrées utilisateur.
4. **Persistance :** L'historique permet de suivre toutes les prédictions.

Limites et améliorations possibles

1. **Dépendance à une seule caractéristique :** L'application n'utilise que UGP2. Ajouter d'autres gènes (par exemple, les 5 gènes avec les plus faibles p-values) pourrait améliorer la robustesse.
2. **Validation des valeurs UGP2 :** Aucune vérification de la plage des valeurs UGP2 (par exemple, valeurs aberrantes). On pourrait ajouter une validation basée sur les données d'entraînement.
3. **Optimisation du modèle :** Les hyperparamètres du modèle ne sont pas optimisés. Une recherche par grille (GridSearchCV) pourrait améliorer les performances.
4. **Performance :** Pour de grands volumes de prédictions, le chargement de l'historique pourrait être lent. Une pagination ou un filtrage pourrait être ajouté.